Doble Grado Ingeniería Informática y Administración y Dirección de Empresas

Sistemas Concurrentes y Distribuidos

Relación de Ejercicios del Tema 1

Ismael Sallami Moreno

22 de octubre de 2024

Índice

1. Ejercicio 1 **2**
1.1. Solución 2
1.2. Análisis 2
1.3. Escenarios posibles 2
1.3.1. Posibles secuencias de ejecución 2
1.3.2. Valores posibles para `x` 3
1.4. Conclusión 3

2. Ejercicio 2 **3**

3. Ejercicio 10 **3**
3.1. Problema de Lógica de Programas Resuelto 4
3.2. Resolución 12 5

4. Ejercicio 14 **6**
4.1. Resolución 14 6

5. Ejercicio 15 **6**
5.1. Resolucion 15 7

6. Ejercicio 22 **8**
6.1. Estudio de Valores Finales 8
6.2. Solución y Explicación Detallada 9
6.3. Resultado Final 9

1. Ejercicio 1

1.Considerar el siguiente fragmento de programa para 2 procesos P1 y P2 Los dos procesos pueden ejecutarse a cualquier velocidad. ¿Cuáles son los posibles valores resultantes para la variable `x`? Suponer que `x` debe ser cargada en un registro para incrementarse y que cada proceso usa un registro diferente para realizar el incremento.

```
{ variables compartidas }

var x : integer := 0 ;
Process P1;                     Process P2;
    var i: integer;                     var j: integer;
begin                           begin
    for i:= 1 to 2 do begin             for i:= 1 to 2 do begin
        x:= x + 1;                              x:= x + 1;
    end                                 end
end
```

1.1. Solución

El problema en la imagen muestra dos procesos, **P1** y **P2**, que están ejecutando un ciclo que incrementa la variable compartida `x` en 1, dos veces cada uno. Ambos procesos pueden ejecutarse a cualquier velocidad, y la variable compartida `x` se debe cargar en un registro antes de ser incrementada. Además, se especifica que cada proceso usa un registro diferente para realizar el incremento, lo cual indica que no hay sincronización inmediata entre los incrementos de los dos procesos.

1.2. Análisis

El incremento `x := x + 1` no es una operación atómica, lo que significa que:

1. El proceso carga el valor de `x` desde la memoria en su propio registro.
2. Luego le suma 1 a ese valor en el registro.
3. Finalmente almacena el valor de vuelta en la variable `x`.

Dado que los procesos pueden ejecutarse a diferentes velocidades, puede ocurrir una **condición de carrera**. Esto significa que los resultados dependen del orden en el que se ejecuten las instrucciones de los dos procesos.

1.3. Escenarios posibles

El valor inicial de `x` es 0, y cada proceso realiza dos incrementos, por lo que hay cuatro incrementos en total. El valor máximo de `x` debe ser 4, pero debido a la falta de atomicidad en el incremento, podrían producirse otros resultados.

1.3.1. Posibles secuencias de ejecución

1. **Ejecución sin interferencia (ideal):**
   - P1 realiza ambos incrementos.
   - P2 realiza ambos incrementos.
   - Resultado: $x = 4$.
2. **Intercalado mínimo:**
   - P1 realiza un incremento.
   - P2 realiza un incremento.
   - P1 realiza el segundo incremento.
   - P2 realiza el segundo incremento.

- Resultado: $x = 4$.

3. **Condición de carrera (interferencias):**
   - P1 y P2 leen el valor de `x` (por ejemplo, ambos leen `x = 0`).
   - Ambos suman 1 en sus respectivos registros.
   - Ambos escriben el valor de vuelta en `x`, pero el valor final de `x` es solo 1.
   - Esto puede ocurrir varias veces, lo que lleva a resultados incorrectos.

1.3.2. Valores posibles para `x`

- En el mejor de los casos, todos los incrementos se realizan correctamente y `x = 4`.
- En el peor de los casos, debido a la interferencia entre los procesos, algunos incrementos pueden sobrescribir los resultados de otros, lo que podría llevar a un valor de `x` de **2** o **3**, si varios incrementos fallan en contabilizarse.

1.4. Conclusión

Los posibles valores de `x` al final de la ejecución de ambos procesos son **2**, **3** o **4**, dependiendo de la intercalación de las instrucciones entre los procesos P1 y P2 debido a la condición de carrera.

2. Ejercicio 2

¿Cómo se podría hacer la copia del fichero `f` en otro `g`, de forma concurrente, utilizando la instrucción concurrente `cobegin-coend`? Para ello, suponer que:

- Los archivos son una secuencia de ítems de un tipo arbitrario `T`, y se encuentran ya abiertos para lectura (`f`) y escritura (`g`). Para leer un ítem de `f` se usa la llamada a función `leer(f)` y para saber si se han leído todos los ítems de `f`, se puede usar la llamada `fin(f)` que devuelve verdadero si ha habido al menos un intento de leer cuando ya no quedan datos. Para escribir un dato `x` en `g` se puede usar la llamada a procedimiento `escribir(g,x)`.
- El orden de los ítems escritos en `g` debe coincidir con el de `f`.
- Dos accesos a dos archivos distintos pueden solaparse en el tiempo.

```
cobegin
    while not fin(f) do
        x := leer(f);
        escribir(g, x);
coend
```

3. Ejercicio 10

Para los siguientes fragmentos de código, obtener la poscondición adecuada para convertirlo en un triple demostrable con la Lógica de Programas:

- (a) {i < 10} i = 2 * i + 1 {}
- (b) {i > 0} i = i - 1; {}
- (c) {i > j} i = i + 1; j = j + 1 {}
- (d) {falso} a = a + 7; {}
- (e) {verdad} i = 3; j = 2 * i {}
- (f) {verdad} c = a + b; c = c / 2 {}

3.1. Problema de Lógica de Programas Resuelto

Para los siguientes fragmentos de código, obtener la poscondición adecuada para convertirlo en un triple demostrable con la Lógica de Programas:

- (a) {i < 10} i = 2 * i + 1 {i < 21}
  - **Precondición:** $i < 10$
  - **Operación:** $i = 2 * i + 1$
  - **Poscondición:** Como inicialmente $i < 10$, al duplicar i y sumar 1, el resultado máximo es $2 * 9 + 1 = 19$, luego $i < 21$.

  Para el cálculo del mismo debemos de calcular la nueva i para la poscondición, $\frac{i-1}{2} < i$. La i de la derecha es la i nueva, asi que la despejamos y nos queda que $i < 21$. Y así con los demás.
- (b) $\{i > 0\}\ i = i - 1\ \{i \geq 0\}$
  - **Precondición:** $i > 0$
  - **Operación:** $i = i - 1$
  - **Poscondición:** Restando 1 a i que es mayor a 0, el nuevo valor de i será mayor o igual a 0.
- (c) $\{i > j\}\ i = i + 1;\ j = j + 1\ \{i > j\}$
  - **Precondición:** $i > j$
  - **Operación:** $i = i + 1;\ j = j + 1$
  - **Poscondición:** Sumando 1 tanto a i como a j no cambia la desigualdad inicial, luego $i > j$.
- (d) {falso} $a = a + 7;$ {verdad}
  - **Precondición:** falso
  - **Operación:** $a = a + 7$
  - **Poscondición:** Cualquier acción derivada de una precondición falsa puede tener cualquier poscondición, por eso obtenemos verdad.
- (e) {verdad} $i = 3;\ j = 2 * i\ \{j = 6\}$
  - **Precondición:** verdad
  - **Operación:** $i = 3;\ j = 2 * i$
  - **Poscondición:** Dado que i se establece en 3, y luego $j = 2 * 3$, entonces $j = 6$.
- (f) {verdad} $c = a + b;\ c = c/2\ \{c = \frac{a+b}{2}\}$
  - **Precondición:** verdad
  - **Operación:** $c = a + b;\ c = c/2$
  - **Poscondición:** Dado que c se establece como la suma de a y b, y luego se divide entre 2, $c = \frac{a+b}{2}$.

Ejercicio 12

Si el triple $\{P\}\ C\ \{Q\}$ es demostrable, indicar por qué los siguientes triples también lo son (o no se pueden demostrar y por qué):

(a) $\{P\}\ C\ \{Q \vee P\}$

(b) $\{P \wedge D\}\ C\ \{Q\}$

(c) $\{P \vee D\}\ C\ \{Q\}$

(d) $\{P\}\ C\ \{Q \vee D\}$

(e) $\{P\}\ C\ \{Q \wedge P\}$

Debemos de tener en cuenta estas dos fórminulas para poder resolver el ejercicio:

- Regla de la consecuencia 1

$$\frac{\{P\}\ S\ \{Q\},\ \{Q\} \rightarrow \{R\}}{\{P\}\ S\ \{R\}} \tag{1}$$

- Regla de la consecuencia 2

$$\frac{\{R\} \rightarrow \{P\},\ \{P\}\ S\ \{Q\}}{\{R\}\ S\ \{Q\}} \tag{2}$$

La lógica que vamos a seguir para resolver este problema es que si hay una disyunción en la poscondición, entonces la poscondición es más débil que la original, por lo que el triple es demostrable. Si hay una conjunción en la poscondición, entonces la poscondición es más fuerte que la original, por lo que el triple no es demostrable. Ya que al ser posible que se de este caso o el otro, es más fácil de que se cumpla.

3.2. Resolución 12

Si el triple $\{P\}\ C\ \{Q\}$ es demostrable, indicar por qué los siguientes triples también lo son (o no se pueden demostrar y por qué):

(a) $\{P\}\ C\ \{Q \vee P\}$

- **Precondición:** P
- **Operación:** C
- **Postcondición:** $Q \vee P$
- **Explicación:** Dado que $Q \rightarrow (Q \vee P)$ es una implicación verdadera, la postcondición se debilita de Q a $Q \vee P$.

(b) $\{P \wedge D\}\ C\ \{Q\}$

- **Precondición:** $P \wedge D$
- **Operación:** C
- **Postcondición:** Q
- **Explicación:** La precondición se fortalece de P a $P \wedge D$, y dado que $P \wedge D \rightarrow P$ es una implicación verdadera, se mantiene la postcondición Q.

(c) $\{P \vee D\}\ C\ \{Q\}$

- **Precondición:** $P \vee D$
- **Operación:** C
- **Postcondición:** Q
- **Explicación:** No es demostrable. La precondición se debilita de P a $P \vee D$, lo cual no garantiza que Q sea verdadero después de C. D puede no estar relacionado con P o Q.

(d) $\{P\}\ C\ \{Q \vee D\}$

- **Precondición:** P
- **Operación:** C
- **Postcondición:** $Q \vee D$
- **Explicación:** Dado que $Q \rightarrow (Q \vee D)$ es una implicación verdadera, la postcondición se debilita de Q a $Q \vee D$.

(e) $\{P\}\ C\ \{Q \wedge P\}$

- **Precondición:** P
- **Operación:** C
- **Postcondición:** $Q \wedge P$
- **Explicación:** No es demostrable. La postcondición se fortalece de Q a $Q \wedge P$, lo cual no garantiza que P siga siendo verdadero después de C, especialmente si C modifica P.

4. Ejercicio 14

Dado el programa `int x = 5, y = 2; cobegin <x = x + y >;<y = x * y >coend;`, obtener:

(a) Valores finales de x e y

(b) Valores finales de x e y si quitamos los símbolos <>de instrucción atómica.

4.1. Resolución 14

Podemos tener los casos simples, es decir, que x=7 e y=14, o que x=15 e y=10. En el caso de que no sean atómicas, el resultado sería x=7 e y=10.

Dado el programa `int x = 5, y = 2; cobegin <x = x + y >;<y = x * y >coend;`, obtener:

(a) **Valores finales de x e y:**

- **Precondición:** $x = 5,\ y = 2$
- **Operación:** $<x = x + y>;\ <y = x * y>$
- **Postcondición:** Como las operaciones dentro de <><>son atómicas, se ejecutan sin interrupciones:
  - Si $x = x + y$ se ejecuta primero, $x = 5 + 2 = 7$. Luego $y = 7 * 2 = 14$.
  - Si $y = x * y$ se ejecuta primero, $y = 5 * 2 = 10$. Luego $x = 5 + 10 = 15$.

  En cualquier orden, el valor final de x e y será diferente. Así que no se puede determinar un único resultado final sin el orden específico de ejecución.

(b) **Valores finales de x e y sin las instrucciones atómicas:**

- **Precondición:** $x = 5,\ y = 2$
- **Operación:** *cobegin* $x = x + y\ ;\ y = x * y$ *coend*
- **Postcondición:** Si las operaciones no son atómicas, pueden interrumpirse entre sí, llevando a diferentes resultados basados en el orden de ejecución:
  - Si $x = x + y$ se ejecuta primero completamente, obtenemos $x = 5 + 2 = 7$. Luego $y = 5 * 2 = 10$. Entonces $x = 7$ y $y = 10$.
  - Si $y = x * y$ se ejecuta primero completamente, obtenemos $y = 5 * 2 = 10$. Luego $x = 5 + 10 = 15$. Entonces $x = 15$ y $y = 10$.

  Pero como no sabemos el orden de las operaciones, también podemos obtener $x = 7$ y $y = 14$. Así que el resultado final puede variar.

5. Ejercicio 15

15. Comprobar si la demostración del triple $\{x \geq 2\}$ $<x = x-2>;$ $\{x \geq 0\}$ interfiere con los teoremas siguientes:

(a) $\{x \geq 0\}$ $<x = x + 3>$ $\{x \geq 3\}$

(b) $\{x \geq 0\}$ $<x = x + 3>$ $\{x \geq 0\}$

(c) $\{x \geq 7\}$ $<x = x + 3>$ $\{x \geq 10\}$

(d) $\{y \geq 0\}$ $<y = y + 3>$ $\{y \geq 3\}$

(e) $\{x \text{ es impar}\}$ $<y = x + 1>$ $\{y \text{ es par}\}$

5.1. Resolucion 15

Advertencia

NO GARANTIZO QUE ESTE BIEN, PERO ES LO QUE HE ENTENDIDO DE LA TEORIA

Comprobar si la demostración del triple $\{x \geq 2\}$ $<x = x - 2>$ $\{x \geq 0\}$ interfiere con los teoremas siguientes:

(a) $\{x \geq 0\}$ $<x = x + 3>$ $\{x \geq 3\}$

- **Precondición:** $x \geq 0$
- **Operación:** $x = x + 3$
- **Postcondición:** $x \geq 3$
- **Explicación:**
  - La precondición indica que antes de ejecutar la instrucción, x es mayor o igual a 0.
  - La operación $x = x + 3$ incrementa el valor de x en 3 unidades.
  - Si x es mayor o igual a 0 antes de la operación, después de sumarle 3, x será mayor o igual a 3.
  - La implicación $Q \to (Q \vee P)$ es siempre verdadera, y dado que $Q = x \geq 3$ y $P = x \geq 0$, la postcondición se debilita de Q a $Q \vee P$.
- **Interferencia:** No hay interferencia porque debilitar la postcondición no afecta la validez del triple.

(b) $\{x \geq 0\}$ $<x = x + 3>$ $\{x \geq 0\}$

- **Precondición:** $x \geq 0$
- **Operación:** $x = x + 3$
- **Postcondición:** $x \geq 0$
- **Explicación:**
  - La precondición indica que antes de ejecutar la instrucción, x es mayor o igual a 0.
  - La operación $x = x + 3$ incrementa el valor de x en 3 unidades.
  - Si x es mayor o igual a 0 antes de la operación, seguirá siendo mayor o igual a 0 después de sumarle 3.
  - La precondición y la postcondición son las mismas.
- **Interferencia:** No hay interferencia porque no hay cambio en las condiciones iniciales ni finales.

(c) $\{x \geq 7\}$ $<x = x + 3>$ $\{x \geq 10\}$

- **Precondición:** $x \geq 7$
- **Operación:** $x = x + 3$
- **Postcondición:** $x \geq 10$
- **Explicación:**
  - La precondición indica que antes de ejecutar la instrucción, x es mayor o igual a 7.
  - La operación $x = x + 3$ incrementa el valor de x en 3 unidades.
  - Si x es mayor o igual a 7 antes de la operación, después de sumarle 3, x será mayor o igual a 10.
  - La precondición se fortalece de $x \geq 2$ a $x \geq 7$, lo que no afecta la postcondición.
- **Interferencia:** No hay interferencia porque fortalecer la precondición no afecta la validez del triple.

(d) $\{y \geq 0\}$ $<y = y + 3>$ $\{y \geq 3\}$

- **Precondición:** $y \geq 0$

- **Operación:** $y = y + 3$
- **Postcondición:** $y \geq 3$
- **Explicación:**
  - La precondición indica que antes de ejecutar la instrucción, y es mayor o igual a 0.
  - La operación $y = y + 3$ incrementa el valor de y en 3 unidades.
  - Si y es mayor o igual a 0 antes de la operación, después de sumarle 3, y será mayor o igual a 3.
  - Aunque se usa una variable diferente (y en lugar de x), el principio sigue siendo el mismo.
- **Interferencia:** No hay interferencia porque el análisis es independiente de la variable usada.

(e) $\{x$ es impar$\}$ $<y = x + 1>$ $\{y$ es par$\}$

- **Precondición:** x es impar
- **Operación:** $y = x + 1$
- **Postcondición:** y es par
- **Explicación:**
  - La precondición indica que antes de ejecutar la instrucción, x es un número impar.
  - La operación $y = x + 1$ incrementa el valor de x en 1 unidad, asignando el resultado a y.
  - Si x es impar, sumar 1 resultará en un número par.
  - La operación $y = x + 1$ garantiza que si x es impar, y será par.
- **Interferencia:** No hay interferencia porque el valor de x después de la instrucción no afecta la postcondición sobre y.

6. Ejercicio 22

Estudiar cuáles son los valores finales de las variables x e y en el siguiente programa secuencial. Insertar los asertos adecuados entre llaves, antes y después de cada sentencia, para poder obtener una traza de demostración del programa, que incluya en su último aserto los valores finales de las variables.

(a) `int x = C1;`

(b) `int y = C2;`

(c) `x = x + y;`

(d) `y = x * y;`

(e) `x = x - y;`

6.1. Estudio de Valores Finales

Estudiar cuáles son los valores finales de las variables x e y en el siguiente programa secuencial. Insertar los asertos adecuados entre llaves, antes y después de cada sentencia, para poder obtener una traza de demostración del programa, que incluya en su último aserto los valores finales de las variables.

(a) `int x = C1;`

(b) `int y = C2;`

(c) `x = x + y;`

(d) `y = x * y;`

(e) `x = x - y;`

6.2. Solución y Explicación Detallada

```
// {x = C1, y = C2}
int x = C1;
// {x = C1, y = C2}
int y = C2;
// {x = C1, y = C2}
x = x + y;
// {x = C1 + C2, y = C2}
y = x * y;
// {x = C1 + C2, y = (C1 + C2) * C2}
x = x - y;
// {x = C1 + C2 - (C1 + C2) * C2, y = (C1 + C2) * C2}
```

- **Inicialización de x:**
  - **Código:** `int x = C1;`
  - **Aserto:** // {x = C1, y = C2}
  - **Explicación:** Inicializamos x con $C1$. En este momento, x es igual a $C1$ y y no está inicializado.
- **Inicialización de y:**
  - **Código:** `int y = C2;`
  - **Aserto:** // {x = C1, y = C2}
  - **Explicación:** Inicializamos y con $C2$. Ahora, x es igual a $C1$ y y es igual a $C2$.
- **Primera Operación (Suma):**
  - **Código:** `x = x + y;`
  - **Aserto:** // {x = C1 + C2, y = C2}
  - **Explicación:** Sumamos y a x y guardamos el resultado en x. Después de esta operación, x es igual a $C1 + C2$ y y sigue siendo $C2$.
- **Segunda Operación (Multiplicación):**
  - **Código:** `y = x * y;`
  - **Aserto:** // {x = C1 + C2, y = (C1 + C2) * C2}
  - **Explicación:** Multiplicamos x por y y guardamos el resultado en y. Después de esta operación, y es igual a $(C1 + C2) * C2$ y x sigue siendo $C1 + C2$.
- **Tercera Operación (Resta):**
  - **Código:** `x = x - y;`
  - **Aserto:** // {x = C1 + C2 - (C1 + C2) * C2, y = (C1 + C2) * C2}
  - **Explicación:** Restamos y de x y guardamos el resultado en x. Después de esta operación, x es igual a $C1 + C2 - (C1 + C2) * C2$ y y sigue siendo $(C1 + C2) * C2$.

6.3. Resultado Final

Después de ejecutar todas las operaciones, los valores finales de las variables son:

- $x = C1 + C2 - (C1 + C2) * C2$
- $y = (C1 + C2) * C2$